

세션 3 · Kafka 운영

Broker · Topic · Partition · Producer · Consumer Group · Offset · 복제 · Retention

대상 버전

Kafka 4.3.x

OS

Ubuntu 24.04 LTS

실습 환경

Azure

01

인프라 구축

03-kafka 스택

노드	사양	역할
kafka-1 kafka-2 kafka-3	Standard_D2ds_v5 2 vCPU / 8 GiB 데이터 디스크 128 GiB	Broker 와 Controller 를 함께 맡음
ops-1	Standard_D2ds_v5 2 vCPU / 8 GiB 데이터 디스크 64 GiB	명령 실행, Prometheus·Grafana

인스턴스 스펙 근거

Kafka 는 메시지를 JVM 힙에 담지 않고 디스크에 쓴 뒤 OS page cache 에서 읽습니다. 메모리는 힙이 아니라 page cache 로 쓰이고, 읽기와 쓰기가 모두 디스크 순차 처리량에 걸립니다.

항목	내용
Standard_D2ds_v5 2 vCPU / 8 GiB	메모리 최적화 계열이 아니라 범용 계열의 최소 크기
OS 와 exporter	약 1 GiB
JVM 힙	2 GiB. 힙을 늘리면 page cache 가 줄어 읽기가 디스크로 내려감
page cache	약 5 GiB. segment 파일을 <code>sendfile</code> 로 소켓에 바로 내보내는 데 씀
데이터 디스크	128 GiB · 3,000 IOPS · 125 MB/s. segment 가 쌓이는 곳

변수 파일 작성

```
subscription_id      = "00000000-0000-0000-0000-000000000000"
resource_group_name  = "rkm-<계정 이름>-kafka-rg"
location              = "koreacentral"
allowed_cidrs         = ["xxx.xxx.xxx.xxx/32"]
ssh_public_key_path  = "~/.ssh/rkm.pub"
```

- **subscription_id**: `az account show --query id -o tsv`
- **resource_group_name**: `rkm-<계정 이름>-kafka-rg`. 형식이 다르면 `apply` 전에 멈춤
- **allowed_cidrs**: `curl -s ifconfig.me` 로 확인한 실습 PC 공인 IP

스택 생성

- 1 `cd lab/terraform/03-kafka`
- 2 `cp session.tfvars.example session.tfvars` 후 값 채우기
- 3 `terraform init`
- 4 `terraform plan -var-file=session.tfvars` 로 자원 수 확인
- 5 `terraform apply -var-file=session.tfvars`
- 6 `terraform output` 으로 ops 노드 공인 IP 확인

Terraform 명령만 실습 PC 에서 실행합니다.

나머지 명령은 ops 노드에서 실행합니다. 마지막 정리에서만 다시 실습 PC 로 돌아옵니다.

완료 판정

`Apply complete!` 로 끝나고 출력 세 개에 값이 전부 채워지면 완료입니다.

생성되는 자원

```
Plan: 27 to add, 0 to change, 0 to destroy.
```

```
Changes to Outputs:
```

```
+ ops_public_ip      = (known after apply)
+ kafka_private_ips  = (known after apply)
+ ssh_command        = (known after apply)
```

- VM 4대: kafka-1 · kafka-2 · kafka-3 과 ops-1
- 데이터 디스크 4개: 서비스 노드 128 GiB, ops 노드 64 GiB
- 네트워크: VNet 하나, 서브넷 하나, 네트워크 보안 그룹 하나. 공인 IP 는 ops-1 에만 붙음

ops 노드 접속

```
ssh -i ~/.ssh/rkm azureuser@<ops 공인 IP>
```

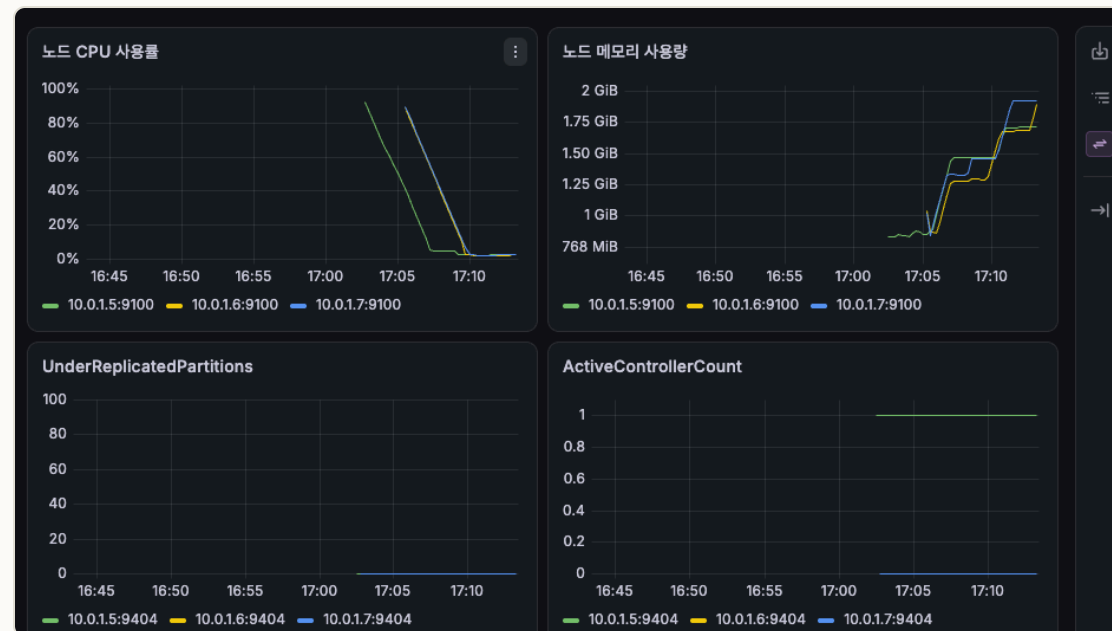
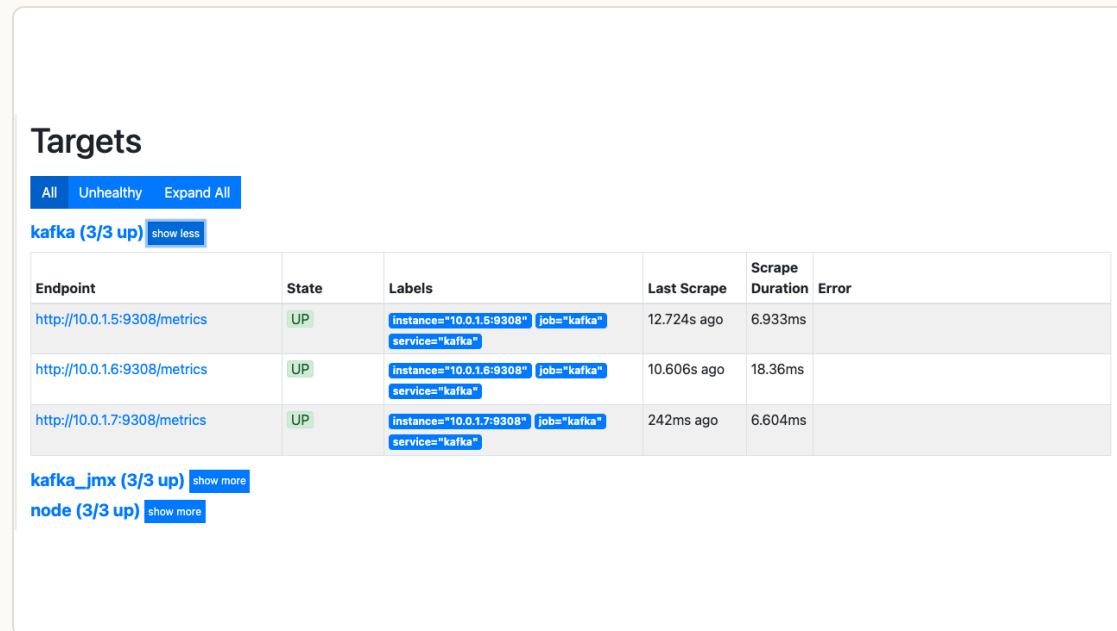

구성 실행

- 1 ops 노드에서 `cd ~/ansible`
- 2 `ansible kafka -m ping` 으로 Broker 3대 연결 확인
- 3 `ansible-playbook play-kafka.yml` 실행
- 4 `ansible kafka -a "systemctl is-active kafka"` 로 기동 확인
- 5 `ansible kafka -a "grep cluster.id /var/lib/kafka/meta.properties"` 로 세 노드의 클러스터 ID 확인
- 6 `source /etc/profile.d/kafka.sh` 로 지금 셸에 Kafka CLI 경로 적용

PLAY RECAP

```
kafka-1  : ok=42  changed=33  unreachable=0  failed=0  skipped=17
kafka-2  : ok=42  changed=32  unreachable=0  failed=0  skipped=17
kafka-3  : ok=42  changed=32  unreachable=0  failed=0  skipped=17
ops-1    : ok=19  changed=15  unreachable=0  failed=0  skipped=26
```

관측 스택 확인



Grafana 대시보드

- **node** : 각 노드의 9100. CPU·메모리·디스크·네트워크
- **kafka** : 각 노드의 9308. Topic·Partition·Consumer Group 지표
- **kafka_jmx** : 각 노드의 9404. 복제 상태와 controller 상태처럼 Broker 프로세스 안에서만 나오는 값

Broker 접속 확인

```
# ops 노드에서
kafka-broker-api-versions.sh --bootstrap-server 10.0.1.5:9092 | grep "id:"

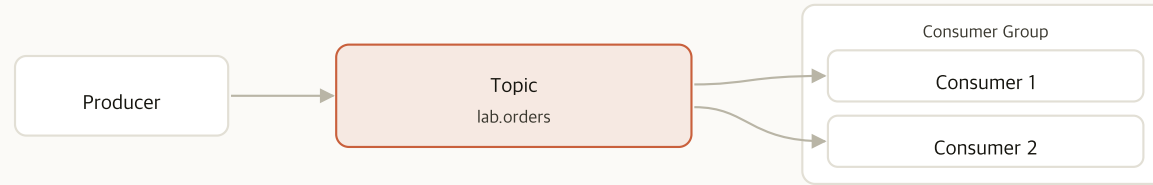
10.0.1.7:9092 (id: 3 rack: null isFenced: false) -> (
10.0.1.6:9092 (id: 2 rack: null isFenced: false) -> (
10.0.1.5:9092 (id: 1 rack: null isFenced: false) -> (
```

- **--bootstrap-server** : 클러스터에 처음 접속할 진입점. 별도의 게이트웨이가 아니라 **Broker 중 하나**
- **한 대만 지정해도 되는 이유**: 모든 Broker 가 클러스터 전체의 메타데이터를 가지고 있어, 첫 응답에 나머지 Broker 주소가 함께 담겨 옴. 이후 통신은 각 Partition 의 leader 와 직접 함
- **운영에서 여러 개를 적는 이유**: 부하 분산이 아니라 적어 둔 한 대가 정지해 있어도 접속되게 하기 위함
- **id** : `node.id` . 인벤토리에 적은 값과 같아야 함

02

주요 구성 요소

주요 구성 요소의 정의



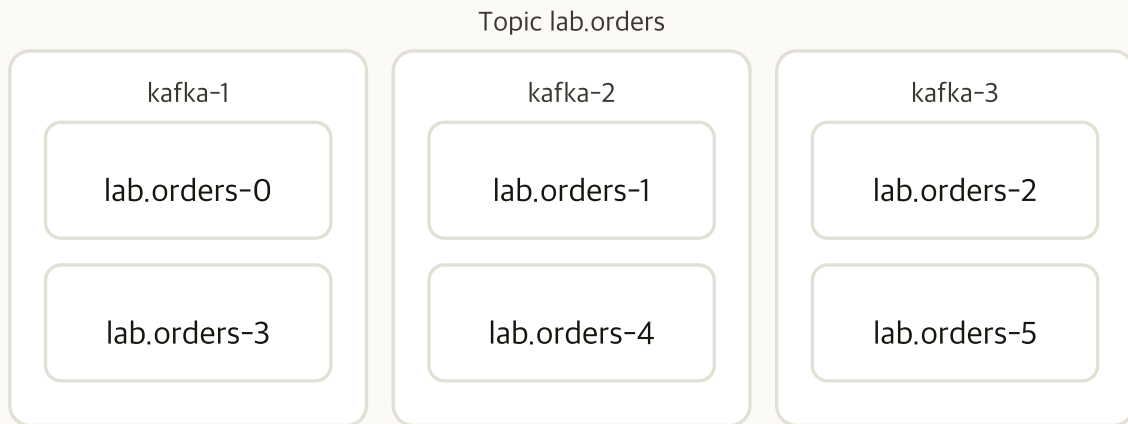
대상	정의
Topic	메시지가 오가는 통로에 붙인 이름. Partition 의 묶음
Producer	Topic 에 메시지를 발행하는 클라이언트
Consumer	Topic 에서 메시지를 읽어 가는 클라이언트
Consumer Group	같은 <code>group.id</code> 로 묶여 Partition 을 나눠 읽는 Consumer 의 집합

Producer 와 Consumer 는 서로를 알지 못합니다.

Broker · Topic · Partition

대상	정의
Broker	Kafka 프로세스 하나이자 노드 하나인 물리 실체
Topic	Partition 의 묶음에 붙인 이름인 논리 단위
Partition	디스크에 실제로 놓이는 로그인 물리 실체

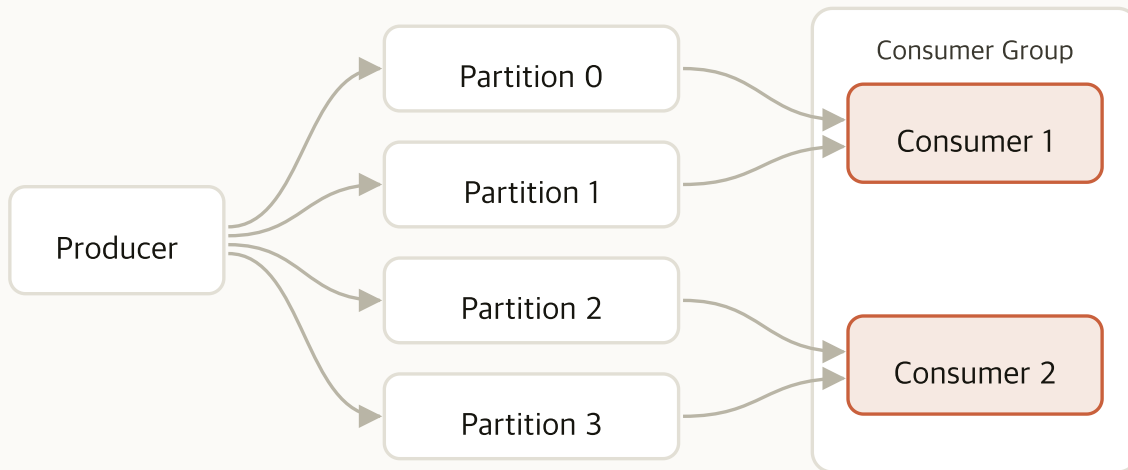
Broker 와 Partition 의 배치



- Topic 1개가 Partition 6개로 구성됨
- 6개의 Partition 이 3개의 Broker 에 배치됨

Topic 에 메시지를 발행(publish) 하였다는 것은 Topic 을 구성하는 Partition 들 중 하나에 메시지가 전달되었다는 의미입니다.

병렬 처리의 단위: Partition



- Partition 1개는 Consumer 1개에 배정됨
- Consumer 1개는 Partition 0개 이상이 배정됨
- Partition 개수보다 Consumer 개수가 많으면, Partition 이 배정되지 않는 Consumer 가 존재함

메시지 순서 보장 범위: Partition



- **순서 보장 범위:** 같은 Partition 내에 전달된 메시지들 간의 처리 순서가 발행 순서로 보장됨
- 서로 다른 Partition 에 전달된 메시지들 간의 처리 순서는 발행 순서로 보장되지 않음
- 순서 의존성이 존재하는 메시지들은 같은 Partition 에 지정되도록 Partitioning 설정

주문 처리 관련된 메시지들이 존재할 때, 같은 주문 ID 를 가지는 메시지 간에만 순서 보장이 되면 됩니다.

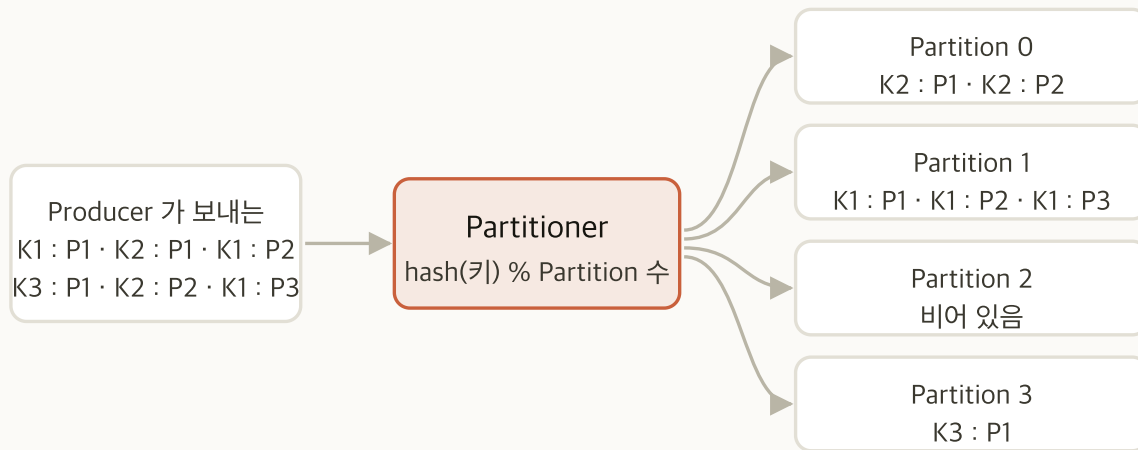
Order 메시지 스트림: (0-1, '접수'), (0-2, '접수'), (0-2, '거절'), (0-1, '수락'), (0-3, '접수'), (0-1, '처리 완료'), (0-3, '수락'), (0-3, '실패')

(0-1, '접수') → (0-1, '수락') → (0-1, '처리 완료')

(0-2, '접수') → (0-2, '거절')

(0-3, '접수') → (0-3, '수락') → (0-3, '실패')

Key 기반 Partitioning



- Kafka 메시지는 Key 와 Payload, Timestamp, Headers 로 구성
- Kafka 메시지의 Key 는 Partitioner 내에서 해시 함수를 통해서 Partition 을 배정받음
- 같은 Key 를 가지는 메시지는 같은 Partition 에 배정되는 것이 보장됨. 따라서 같은 Key 를 가지는 메시지의 처리 순서는 발행 된 순서에 따르는 것이 보장됨
- Key 가 없는 메시지들은 Partition 들에 고르게 흩어져서 전달됨

Partition 수 결정

- Partition 수는 한 번 정하면 변경하기 어려움
- Partition 개수 줄이는 것은 불가능
- Partition 개수를 늘리면 새로운 메시지의 Key 가 과거 메시지의 Key 와 같은 값을 가지더라도 같은 Partition 에 배정되는 것을 보장하지 못함
- 병렬성 목표에 따라 개수 설정하되, Broker 수의 배수로 잡으면 고르게 분포됨

Topic 과 RF

- RF (**replication.factor**): 하나의 Partition 을 몇 개의 Broker 에 복제할 것인가
- RF 는 Broker 의 수를 넘을 수 없음: RF 가 Broker 의 수를 넘으면 같은 Broker 에 같은 Partition 복제본이 여럿 배치된다는 것이므로 가용성의 의미를 갖지 못함

Topic 생성

ops 노드에서 실행합니다.

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --create --topic lab.orders --partitions 6 --replication-factor 3`
- 2 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic lab.orders` 로 배치 확인
- 3 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --create --topic lab.rf4 --partitions 1 --replication-factor 4` 로 거부되는 것 확인

RF=4 로 설정된 Topic 생성은 RF 가 Broker 개수(3개)를 초과했으므로 거부됩니다.

Key 와 Partition 배정 확인

ops 노드에서 실행합니다.

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --create --topic lab.keys --partitions 3 --replication-factor 3`
- 2 `kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.keys --reader-property parse.key=true --reader-property key.separator=:` 실행 후 `A:1` `D:1` `F:1` `A:2` `D:2` 를 한 줄씩 입력하고 종료
- 3 `for i in 0 1 2; do echo "-- partition $i --"; kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.keys --partition $i --from-beginning --formatter-property print.key=true --timeout-ms 3000; done`
- 4 어느 키가 어느 Partition 에 있는지 기록

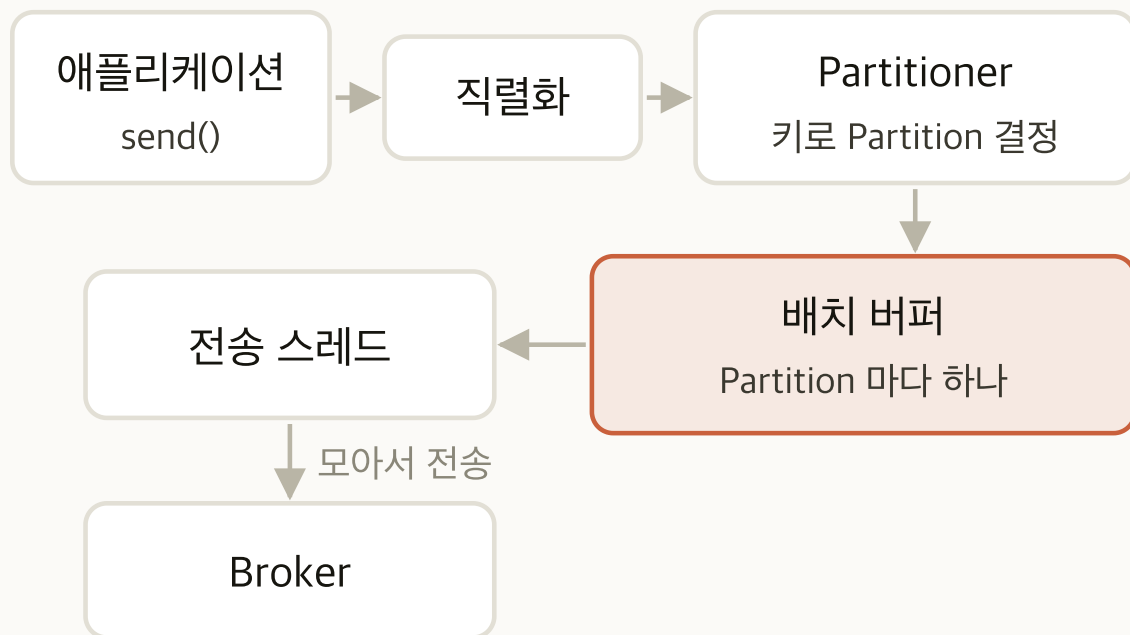
Key 값으로 `A` 를 가지는 두 메시지가 같은 Partition 에 배정되어야 합니다.

Partition 수 증가

ops 노드에서 실행합니다.

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --alter --topic lab.keys --partitions 4` 로 Partition 수를 3개에서 4개로 변경
- 2 `kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.keys --reader-property parse.key=true --reader-property key.separator=:` 실행 후 `A:3` `D:3` `F:3` 입력
- 3 `for i in 0 1 2 3; do echo "-- partition $i --"; kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.keys --partition $i --from-beginning --formatter-property print.key=true --timeout-ms 3000; done`
- 4 `A` 의 3번째 메시지가 앞의 둘과 다른 Partition 에 있는지 확인
- 5 줄이는 시도: 거부됨. `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --alter --topic lab.keys --partitions 2`

Producer 의 전송 경로



- **send()** : 메시지를 버퍼에 넣은 뒤 반환됨
- 버퍼는 Partition 마다 하나씩 존재
- Partitioner 에서 메시지의 Partition 을 배정하여 버퍼에 넣음
- 전송 스레드는 버퍼의 메시지를 배치로 Broker 에 전송

Producer 배치 설정

Producer 는 Partition 마다 존재하는 배치 버퍼에 메시지를 모아 두었다가 전송합니다.
 다음 설정들은 모아 둔 메시지를 보내는 기준에 대한 설정입니다.

설정	정의
<code>batch.size</code>	Partition 하나의 배치가 이 크기에 차면 곧바로 보냄
<code>linger.ms</code>	배치가 다 차지 않았을 때 더 모으려고 기다리는 최대 시간
<code>buffer.memory</code>	Partition 별 배치 버퍼를 전부 합쳐 쓸 수 있는 메모리 총량

메시지 전송 멍등성

상황	Producer 동작	결과
멍등성 꺼짐 · Broker 가 받고 응답 직전에 끊김	재시도	같은 메시지가 두 번 저장됨
멍등성 켜짐 · 같은 상황	Producer ID 와 시퀀스 번호를 함께 보냄	Broker 가 중복을 걸러냄

- **설정 이름:** `enable.idempotence` . Producer 설정
- **기본 활성화:** 3.0 부터 기본값이 `true`

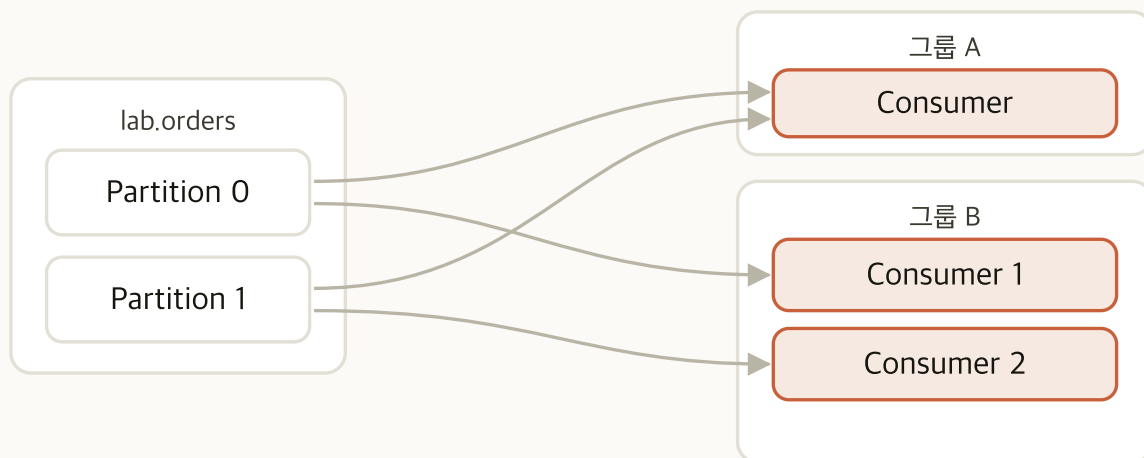
메시지 압축

값	압축률	특징
<code>none</code>	없음	CPU 를 쓰지 않음
<code>gzip</code>	높음	CPU 를 많이 씀
<code>snappy</code>	낮음	빠름
<code>lz4</code>	중간	빠르고 균형이 좋음
<code>zstd</code>	높음	압축률과 속도가 모두 좋음. 최근의 기본 선택

- **Producer 가 압축:** 배치 단위로 압축
- **Broker 는 그대로 저장:** 풀지 않음. 디스크와 네트워크를 함께 아낌
- **Consumer 가 압축 해제:** 양쪽 클라이언트가 CPU 를 씀

Consumer Group

하나의 Topic 을 여러 Consumer 가 나눠 처리하면서도 같은 메시지를 중복해서 읽지 않으려면, 어느 Consumer 가 어느 Partition 을 맡을지 정해 주는 단위가 필요합니다. 그 단위가 Consumer Group 입니다.



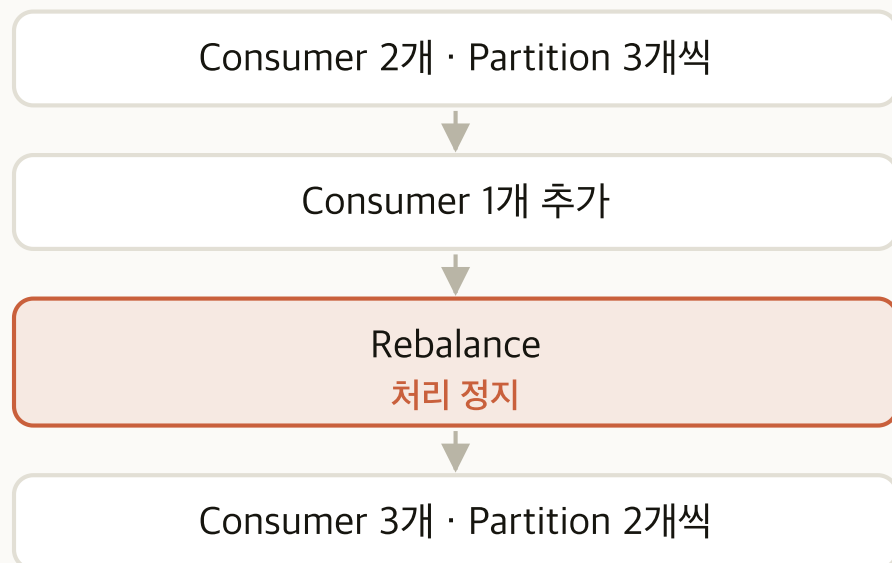
- Partition 1개는 Consumer 1개에 의해서만 소비됨
- Group 간의 Partition 소비는 독립적임

group.id 가 같으면 나눠 읽고, 다르면 각자 다 읽습니다.

같은 메시지 스트림을 서로 다른 애플리케이션이 소비할 때, 애플리케이션별로 그룹을 나눕니다.

Rebalance

Rebalance 는 그룹 안의 Consumer 들에게 Partition 을 다시 나눠 주는 것입니다.
 멤버가 바뀌면 누가 어느 Partition 을 맡을지 처음부터 다시 정합니다.



- **트리거**: Consumer Group 내 Consumer 멤버 추가·이탈, Partition 수 변경
- **부작용**: Rebalance 로 인한 Partition 재할당이 끝날 때까지 처리 멈춤

Rebalance 가 잦으면 처리가 밀리게 되므로 최소화해야 합니다.

Rebalance 시 Partition 할당 전략

Rebalance 가 일어날 때, Partition 을 Consumer Group 내의 Consumer 들에게 할당하는 전략 설정들입니다.

전략	할당 방식	Rebalance 중
Range	Topic 마다 그 Topic 의 Partition 을 번호순으로 늘어놓고, 구독 중인 Consumer 에게 앞에서부터 연속된 구간으로 잘라 줌	전원 정지 (eager)
CooperativeSticky	기존 할당을 유지하며 필요한 것만 옮김	옮기지 않는 Consumer 는 계속 처리 (cooperative)

이전부터 기본값이었던 Range 가 하위 호환을 위해 기본값으로 설정되어 있으나, 무중단 처리를 위해 CooperativeSticky 사용이 권장됩니다.

Consumer liveness 판정

Broker 는 Consumer 가 살아 있는지를 하트비트와 `poll()` 간격 두 가지 조건으로 판정하고, 둘 중 하나만 불만족해도 Consumer Group 에서 제외합니다.

Consumer 가 Consumer Group 에서 제외되면 Rebalance 가 동작합니다.

설정	정의
<code>heartbeat.interval.ms</code>	하트비트를 보내는 주기
<code>session.timeout.ms</code>	하트비트를 이 시간 동안 못 받으면 정지로 판정하는 한계
<code>max.poll.interval.ms</code>	<code>poll()</code> 사이에 허용되는 최대 간격

`poll()` 은 Consumer 가 Broker 에 메시지 배치를 받아 오는 호출입니다.

`poll()` 호출하여 받아온 메시지 배치 처리가 길어지면 다음 `poll()` 호출 간격이 `max.poll.interval.ms` 보다 길어져서 Consumer Group 제외로 이어질 수 있습니다.

Consumer 의 static membership 설정

Consumer 애플리케이션을 배포하는 과정에서 불필요한 Rebalance 가 일어나지 않게 하는 설정입니다.

Consumer 가 재시작 동안에 Consumer Group 을 짧게 이탈했다가 복귀하는 과정에서 Rebalance 를 막기 위해서 사용할 수 있습니다.

- Consumer 에 고정 ID(`group.instance.id`)를 붙임
- 고정 ID 가 부여된 Consumer 에 대해서는 이탈이 발생해도 `session.timeout.ms` 시간 동안 Rebalance 를 하지 않고 기다림

기본으로 켜는 설정은 아닙니다.

고정 ID 를 부여하여 관리해야 하고, 의도하지 않은 Consumer 정지가 발생할 경우 감지가 늦을 수 있습니다.

Consumer Group 구성

ops 노드에서 실행합니다.

- 1 터미널 두 개를 열고 각각


```
kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --group lab-cg --formatter-property print.partition=true --command-property client.id=consumer-1
```

```
kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --group lab-cg --formatter-property print.partition=true --command-property client.id=consumer-2
```

 실행
- 2 세 번째 터미널을 열고 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --describe --group lab-cg` 로 Consumer 별 Partition 할당 확인
- 3 세 번째 터미널에서 `kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --reader-property parse.key=true --reader-property key.separator=:` 실행 후 `A:1 B:1 A:2 C:1 A:3 B:2 C:2` 를 한 줄씩 차례대로 입력하고 종료
- 4 Consumer 를 실행한 두 터미널의 출력에서 Partition 번호 및 메시지 확인
- 5 네 번째 터미널을 열고 `kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --group lab-cg --formatter-property print.partition=true --command-property client.id=consumer-3` 을 실행하여 consumer-3 을 생성하고, 세 번째 터미널에서 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --describe --group lab-cg` 로 Consumer 별 Partition 할당 재확인

--describe --group 출력

```
GROUP      TOPIC      PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  CONSUMER-ID
lab-cg     lab.orders 0           3               3               0    consumer-1-...
lab-cg     lab.orders 1           2               2               0    consumer-1-...
lab-cg     lab.orders 2           0               0               0    consumer-1-...
lab-cg     lab.orders 3           2               2               0    consumer-2-...
lab-cg     lab.orders 4           0               0               0    consumer-2-...
lab-cg     lab.orders 5           0               0               0    consumer-2-...
```

03

Offset

Offset



- **정의:** Partition 안에서 메시지에 붙는 단조 증가 번호
- 하나의 Partition 안에서만 의미 있음. Topic 전체를 아우르는 번호는 없음
- Consumer Group 별로 offset commit 위치를 별도로 가지며, 이 위치를 마음대로 조정할 수 있음
- Offset commit 위치를 앞으로 당기면 해당 offset 위치부터 메시지를 재처리 할 수 있음

Offset 의 저장 위치와 형태

commit 된 Offset 은 Consumer Group 과 Partition 의 조합마다 하나씩 존재하고, Kafka 의 내부 Topic `__consumer_offsets` 에 저장됩니다.

group.id	topic	partition	다음에 읽을 위치
lab-cg	lab.orders	0	152
lab-cg	lab.orders	1	148
lab-cg	lab.orders	2	150
analytics	lab.orders	0	40

- 키: `group.id` · `topic` · `partition` 세 값의 조합. 같은 Topic 을 읽어도 그룹이 다르면 값이 따로 존재
- 저장 방식: `cleanup.policy=compact` 라 키마다 최신 값 하나만 남음

Offset commit 시점과 전달 보장

Offset commit 은 Consumer Group 이 어느 Partition 을 어디까지 처리했는지를 Broker 에 기록하는 것입니다. 이 기록이 다음에 읽기 시작할 위치가 되므로, 언제 기록하느냐에 따라 장애가 났을 때 메시지가 빠지기도 하고 겹치기도 합니다.



- **처리 전 commit:** 처리 중에 정지하면 그 메시지 처리 기회를 잃음
- **처리 후 commit:** commit 전에 정지하면 다시 처리함
- **auto commit:** `auto.commit.interval.ms` 마다 자동으로 커밋함. 처리가 끝났는지와 무관함

auto.offset.reset

값	커밋된 Offset 이 없을 때	언제 쓰는가
latest (기본)	지금부터 들어오는 것만 읽음	과거가 필요 없는 실시간 처리
earliest	남아 있는 가장 오래된 것부터 읽음	처음부터 다시 처리해야 하는 경우
none	예외를 냄	Offset 이 없는 상황 자체를 오류로 다뤄야 할 때

기본값이 **latest** 이므로, 새 Consumer Group 은 접속 이전에 쌓인 메시지를 읽지 않습니다.

Consumer Group Offset 조정

`kafka-consumer-groups.sh --reset-offsets` 는 특정 Consumer Group 의 처리 위치를 사람이 직접 옮기는 명령입니다. Topic 의 메시지는 변경하지 않고 그 그룹의 위치만 바꿉니다.

옵션	조정 위치	쓰는 곳
<code>--to-earliest</code>	남아 있는 가장 오래된 위치	전체 재처리
<code>--to-latest</code>	마지막 위치	밀린 것을 버리고 지금부터
<code>--to-offset <N></code> · <code>--shift-by <N></code>	지정한 Offset · 현재 위치에서 앞뒤로	특정 지점부터 다시. 조금만 되감을 때
<code>--to-datetime <시각></code>	그 시각 이후 첫 메시지	장애 발생 시각부터 재처리

Offset 되감기

ops 노드에서 실행합니다.

- 1 `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic __consumer_offsets | head -1` 로 정책이 `compact` 인 것 확인
- 2 Consumer 를 전부 정지. `pkill -f ConsoleConsumer` . 정지 뒤 그룹이 `Empty` 가 될 때까지 최대 45초 기다림
- 3 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --describe --group lab-cg` 로 현재 `CURRENT-OFFSET` 을 기록
- 4 `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --group lab-cg --topic lab.orders --reset-offsets --to-earliest --dry-run`
- 5 출력의 `NEW-OFFSET` 열 확인. **아직 반영되지 않았습니다**
- 6 같은 명령에서 `--dry-run` 을 `--execute` 로 바꿔 실행
- 7 `--describe` 로 `CURRENT-OFFSET` 이 0이 된 것 확인
- 8 Consumer 재기동. `kafka-console-consumer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --group lab-cg --formatter-property print.partition=true --command-property client.id=consumer-1`

--dry-run 과 --execute

```
GROUP    TOPIC      PARTITION  NEW-OFFSET
lab-cg    lab.orders    0          0
lab-cg    lab.orders    1          0
lab-cg    lab.orders    2          0
... Partition 3·4·5 도 같은 값입니다
```

- `--dry-run` 과 `--execute` 의 출력이 같음: 실제로 반영됐는지는 `--describe` 로 확인
- `NEW-OFFSET` 이 0: `--to-earliest` 이고 아직 아무것도 삭제되지 않음

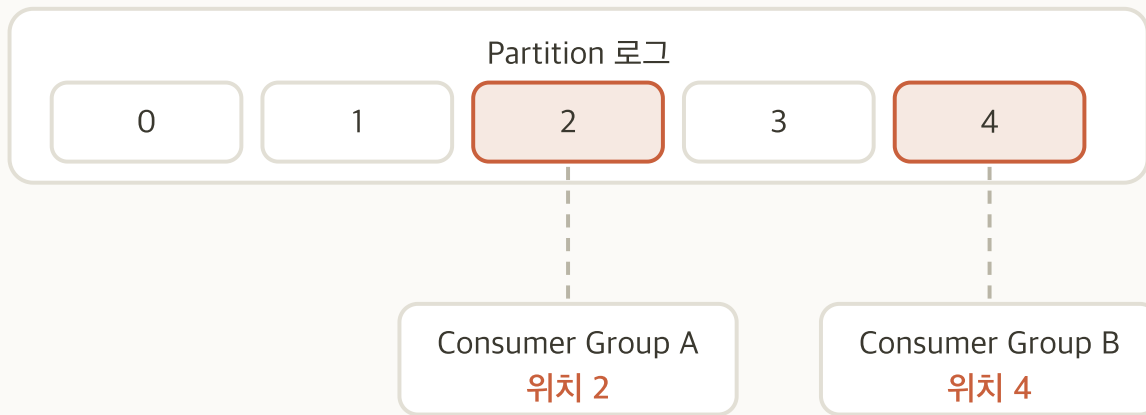
`--to-earliest` 는 0이 아니라 「남아 있는 가장 오래된 것」입니다.

04

로그 기반 메시징과 저장 구조

로그 기반 메시징

Kafka 는 메시지를 **끝에만 덧붙이는 로그 파일**에 기록합니다. 읽은 메시지를 지우지 않고, 각 Consumer Group 이 자신의 처리 위치를 따로 관리합니다.



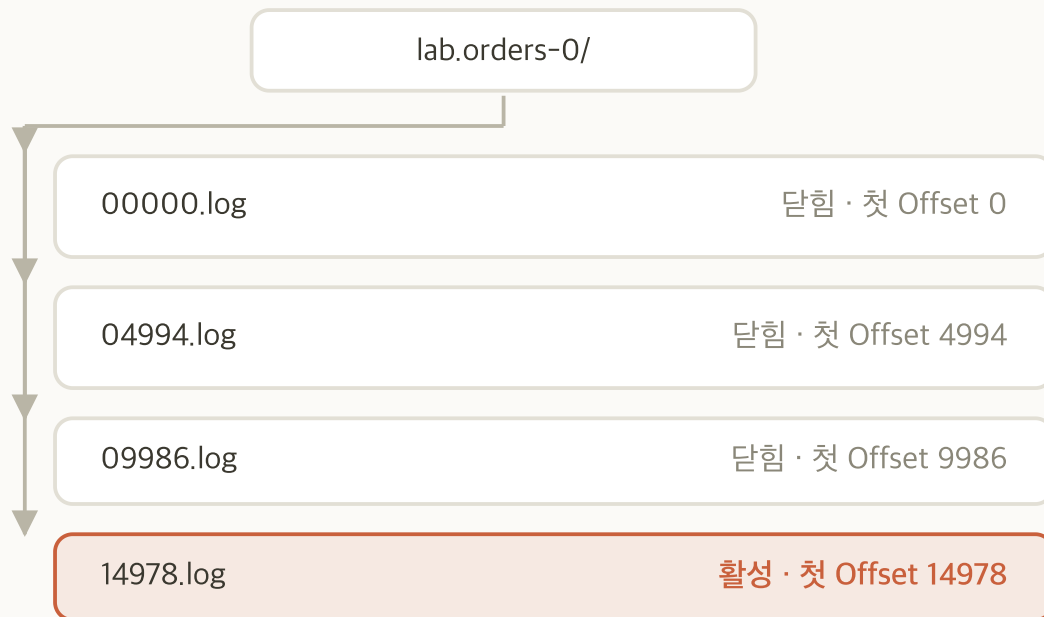
- **읽어도 삭제되지 않음**: 처리 위치만 이동함
- **Consumer Group 간 독립**: 서로의 처리 위치에 영향을 주지 않음
- **재처리 가능**: 처리 위치를 앞으로 되돌리면 같은 메시지를 다시 읽음

큐와 비교했을 때 가장 차이나는 점입니다.

큐는 메시지를 꺼내면 사라지지만, 로그는 그대로 남고 보관 기간은 Retention 이 정합니다.

segment 파일

segment 는 Partition 의 로그를 일정 크기나 기간 단위로 끊어 저장한 파일입니다.



- **롤링:** `log.segment.bytes` 를 넘거나 `log.roll.ms` 가 지나면 새 파일이 열림. Topic 단위 설정은 `segment.bytes` · `segment.ms`
- **단함:** 이전 파일에는 기록하지 않음. 내용이 고정되고 **마지막 메시지 시각이 확정됨**
- **활성 segment:** 지금 기록 중인 파일 하나. 기록이 이어지는 동안 시각이 갱신되어 **삭제 판정 대상이 아님**

메시지 단위 삭제는 불가능합니다. 중간을 삭제하면 뒤따르는 Offset 이 전부 이동하기 때문입니다.

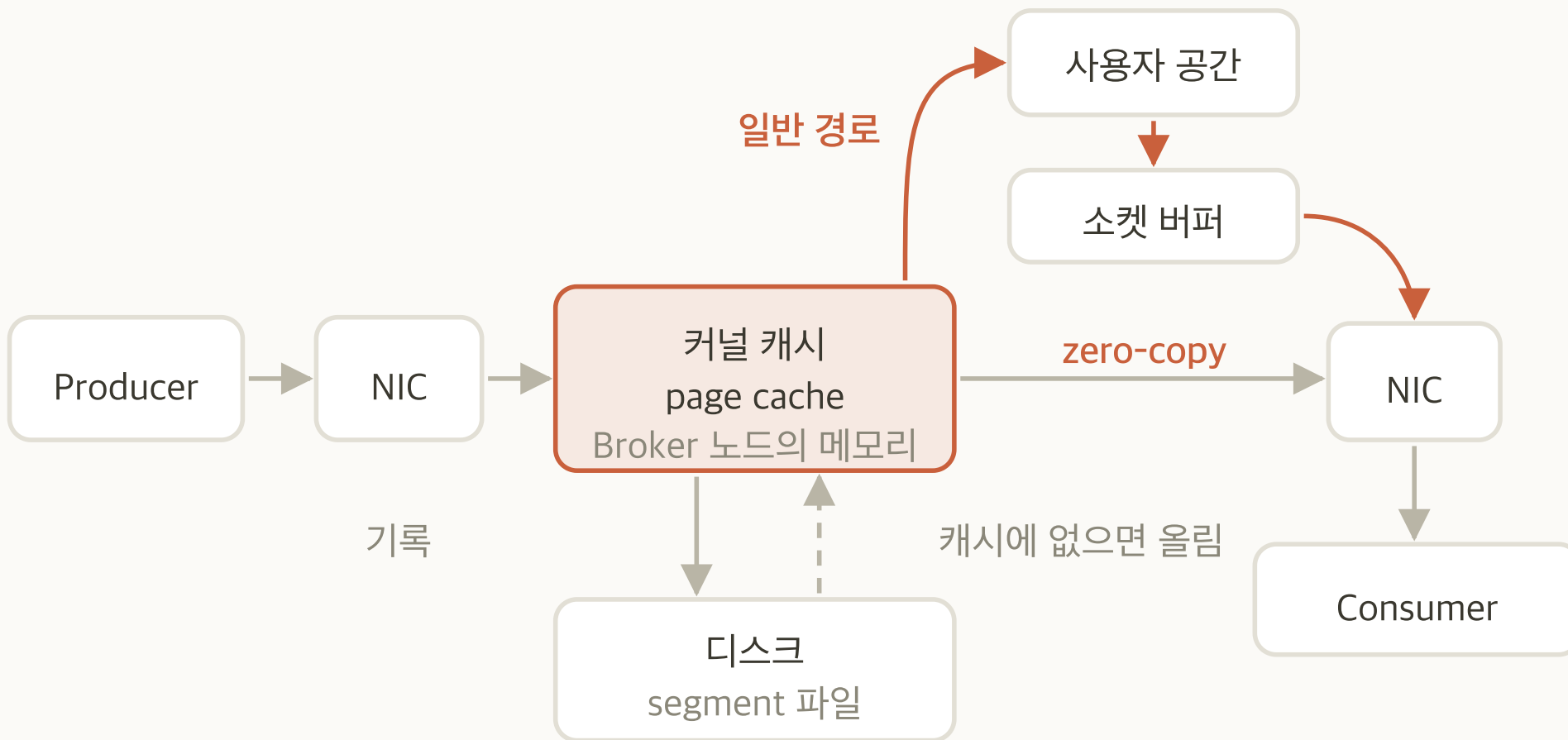
디스크가 병목이 아닌 구조

기법	동작	효과
순차 IO	파일 끝에만 기록하고 순서대로 읽음	디스크 탐색이 거의 발생하지 않음
OS 캐시	기록 직후의 데이터가 캐시에 남아 있음	최근 데이터를 읽는 Consumer 는 디스크에 접근하지 않음
zero-copy (<code>sendfile</code>)	커널이 파일에서 소켓으로 바로 보냄	메시지가 사용자 공간과 JVM 힙을 거치지 않음

세 기법 모두 메시지를 JVM 힙에 적재하는 방식이 아닙니다.

Broker 는 Java 로 구현되어 JVM 위에서 동작하지만, 힙은 요청 버퍼와 메타데이터에만 쓰이고 메시지는 OS 캐시가 담당합니다.

메시지의 이동 경로



두 복사 경로의 대비

page cache 는 커널이 디스크 내용을 보관하는 Broker 노드의 메모리이고, NIC 는 네트워크 카드입니다. 파일을 네트워크로 전송하려면 이 둘 사이를 거쳐야 합니다.

항목	일반 경로	zero-copy
거치는 곳	page cache → 사용자 공간 버퍼 → 소켓 버퍼 → NIC	page cache → NIC
복사 횟수	4회	2회
시스템 호출	2회 (read 와 write)	1회 (sendfile)
CPU 의 역할	버퍼 사이를 두 차례 복사	거의 없음

디스크가 빨라서가 아니라 거치는 단계가 적기 때문입니다.

Kafka 는 메시지를 가공하지 않고 그대로 전송하므로 사용자 공간으로 올릴 필요가 없습니다. Broker 가 메시지 내용을 다루지 않는다는 것이 이 구조의 전제입니다.

디스크 성능 지표와 Kafka 의 병목

지표	측정 대상	Kafka 에서의 의미
IOPS	초당 처리하는 IO 요청 수	Kafka 는 segment 끝에만 이어서 기록함. 흩어진 위치에 접근하지 않으므로 큰 덩어리 하나가 요청 하나가 되고, 같은 양을 전송하는 데 필요한 요청 수가 적음
처리량 (MB/s)	초당 전송하는 바이트	여기가 상한. 요청 수가 아니라 전송량이 먼저 한계에 도달함
지연	요청 하나의 응답 시간	최근 데이터는 OS 캐시에서 응답하므로 디스크까지 내려가지 않음. 디스크 지연을 낮춰도 효과가 작고, 캐시를 벗어난 읽기에서만 그대로 드러남

디스크를 선정할 때 기준은 순차 처리량입니다.

랜덤 IOPS 를 올려도 효과가 작습니다. 흩어진 위치를 읽는 MongoDB 와 반대입니다.

뒤쳐진 Consumer 가 미치는 영향

OS 캐시는 Broker 노드의 메모리에 있습니다. 디스크에서 읽은 파일 내용을 커널이 메모리에 보관하다가 다음 요청에 그대로 응답하는 구조이며, 공간이 부족하면 오래 참조되지 않은 것부터 제거합니다.

- ① 최근 데이터는 이미 메모리에 있음: 기록 직후라 캐시에 남아 있어 Consumer 가 디스크에 접근하지 않음
- ② 오래된 Offset 을 읽으면 디스크에서 적재해야 함: 캐시에 없으므로 그만큼 느림
- ③ 그 과정에서 최근 데이터가 캐시에서 밀려남: 커널이 공간을 확보하려고 오래 참조되지 않은 것부터 제거함
- ④ 최근 데이터를 읽던 Consumer 도 디스크로 내려감: 그 Consumer 는 조건이 달라지지 않았는데도 읽기가 느려짐

뒤쳐진 Consumer 하나의 영향이 그 Consumer 에 그치지 않습니다.

오래된 구간을 훑는 동안 같은 Broker 를 읽는 다른 Consumer 의 읽기까지 디스크로 내려갑니다.

05

Retention 과 segment

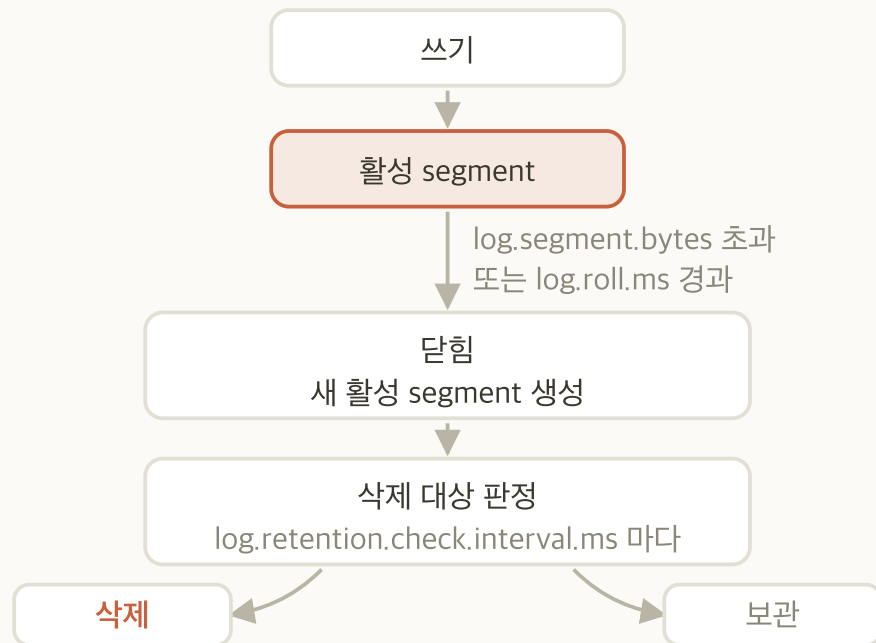
Retention 의 두 기준

Retention 은 Broker 가 메시지를 언제까지 디스크에 남겨 둘지 정하는 규칙입니다.

Consumer 가 읽었는지와 무관하게, 정해진 기준을 넘긴 segment 를 Broker 가 스스로 지웁니다.

설정	기준	결지 않으면
<code>retention.ms</code>	시간. segment 의 마지막 메시지 시각 기준	기본 7일
<code>retention.bytes</code>	Partition 당 용량	기본이 무제한. 결지 않으면 디스크가 참
둘 다 걸면	먼저 도달하는 쪽이 적용됨	어느 쪽이 걸렸는지 로그로 확인

segment 롤링과 삭제



- **롤링**: 활성 segment 가 `segment.bytes` 를 넘거나 `segment.ms` 가 지나면 그 파일이 닫히고 새 파일이 열리는 것
- **활성 segment 의 삭제 제외**: 닫혀야 삭제 판정에 들어감
- **판정 주기**: `log.retention.check.interval.ms` 마다. 제품 기본값은 5분이고 이 실습은 1분으로 낮춰 두었음
- **그래서 즉시 지워지지 않음**: 롤링과 주기 둘을 기다려야 함

Retention 을 줄여도 디스크가 곧바로 비지 않습니다.

적재가 계속되면 활성 segment 는 남습니다. 적재가 멈추면 활성 segment 도 닫혀 삭제 대상이 되고, 새 빈 segment 가 하나 열립니다.

cleanup.policy

값	무엇을 남기는가	쓰는 곳
delete (기본)	오래된 segment 부터 지움	이벤트 스트림. 지나간 것은 필요 없음
compact	Key 별 최신 값만 남김	상태 저장. <code>__consumer_offsets</code> 가 이 방식
compact,delete	압축하고 오래된 것도 지움	상태를 두되 무한히 보관하지는 않을 때

compact 는 Key 가 있고, 한 Key 에 같은 종류의 메시지만 담을 때 성립합니다.

주문 ID 를 Key 로 두고 주문 메타데이터와 상태 변경, 상세 항목을 섞어 보내면 그중 최신 하나만 남고 나머지는 사라집니다.

디스크 포화

Retention 을 걸지 않으면 segment 가 지워지지 않고 계속 쌓입니다.

디스크가 차면 새 segment 를 열지도, 활성 segment 에 이어 쓰지도 못해 그 Broker 의 쓰기가 멈춥니다.

- **쓰기 실패:** 그 Broker 의 Partition 이 전부 멈춤
- **복제 중단:** 그 Broker 를 따라오던 복제본이 더 이상 받지 못해 뒤처짐
- **Retention 을 줄여도 곧바로 반영되지 않음:** 롤링과 판정 주기를 기다려야 함
- **사전 설정:** `retention.bytes` 를 처음부터 걸어 둠

디스크가 차기 시작한 뒤에 Retention 을 줄이는 것은 늦습니다.

지워질 때까지 시간이 걸리고 그동안 쓰기는 계속 실패합니다.

segment 생성

ops 노드에서 실행합니다.

- 1 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config segment.bytes=1048576` 로 segment 를 1 MiB 로
- 2 **Consumer 를 먼저 정지.** `pkill -f ConsoleConsumer` . 정지하지 않으면 10만 건이 화면에 그대로 출력됩니다
- 3 `kafka-producer-perf-test.sh --topic lab.orders --num-records 100000 --record-size 200 --throughput -1 --command-property bootstrap.servers=10.0.1.5:9092` 로 적재
- 4 `ansible kafka -a "ls -l /var/lib/kafka/lab.orders-0"` 로 segment 파일 목록 확인
- 5 **파일 이름과 개수를 기록**

완료 판정

`.log` 파일이 여러 개 보이면 완료입니다.

Partition 디렉터리

```
-rw-r--r-- 1048274 00000000000000000000.log
-rw-r--r-- 1048128 000000000000000004994.log
-rw-r--r-- 1048128 000000000000000009986.log
-rw-r--r-- 294786 000000000000000014978.log <- 활성
-rw-r--r-- 10485760 000000000000000014978.index
... segment 마다 .index · .timeindex · .snapshot 이 함께 있습니다
```

- 파일 이름은 첫 Offset: `000000000000000004994.log` 는 4994번부터 담음
- 닫힌 segment 는 `segment.bytes` 에 가까움: 활성 segment 만 작음
- `.index` · `.timeindex` 가 함께 있음: 활성 segment 의 인덱스는 미리 크게 잡혀 있음
- 디렉터리에는 이 밖에도 있음: `leader-epoch-checkpoint` 와 `partition.metadata` 가 함께 놓임

Retention 조정과 삭제 확인

- 1 ops 노드에서. `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config retention.ms=60000` 으로 Retention 을 1분으로
- 2 곧바로 `ansible kafka -a "ls /var/lib/kafka/lab.orders-0"` 실행. 파일이 그대로인 것 확인
- 3 판정 주기가 지난 뒤 같은 명령을 다시 실행. **최대 2분입니다**
- 4 오래된 segment 부터 사라졌는지, 새 빈 segment 하나만 남았는지 확인
- 5 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --delete-config segment.bytes,retention.ms` 로 되돌리고 `--describe --entity-type topics --entity-name lab.orders` 로 동적 설정이 비었는지 확인
- 6 Consumer 는 앞 단계에서 이미 정지했습니다. `kafka-consumer-groups.sh --bootstrap-server 10.0.1.5:9092 --group lab-cg --topic lab.orders --reset-offsets --to-earliest --dry-run` 으로 `NEW-OFFSET` 이 00이 아닌 것 확인

여기서 거는 Retention 은 이미 쌓인 segment 를 영구 삭제합니다.
삭제된 것은 복구할 수 없습니다. 남길 출력은 먼저 기록해 두세요.

삭제 전후 대비

```
# retention.ms 조정 직후. .log 가 넷
...000000.log ...004994.log ...009986.log ...014978.log
# 판정 주기가 지난 뒤. 넷 다 .deleted 로 바뀌고
...016382.log <- 새로 열린 빈 segment 하나만 남습니다
```

- **삭제 순서:** 오래된 것부터. 앞 Offset 의 파일이 먼저 지워짐
- **적재가 멈추면 활성 segment 도 닫히고 지워짐:** 새 빈 segment 하나만 남음
- **조정 직후에는 그대로:** 판정 주기를 기다려야 함

--to-earliest 가 가리키는 위치가 이 결과로 달라집니다.

앞부분이 지워졌으므로 「남아 있는 가장 오래된 위치」가 0이 아닙니다.

06

KRaft 와 클러스터 메타데이터

클러스터 메타데이터

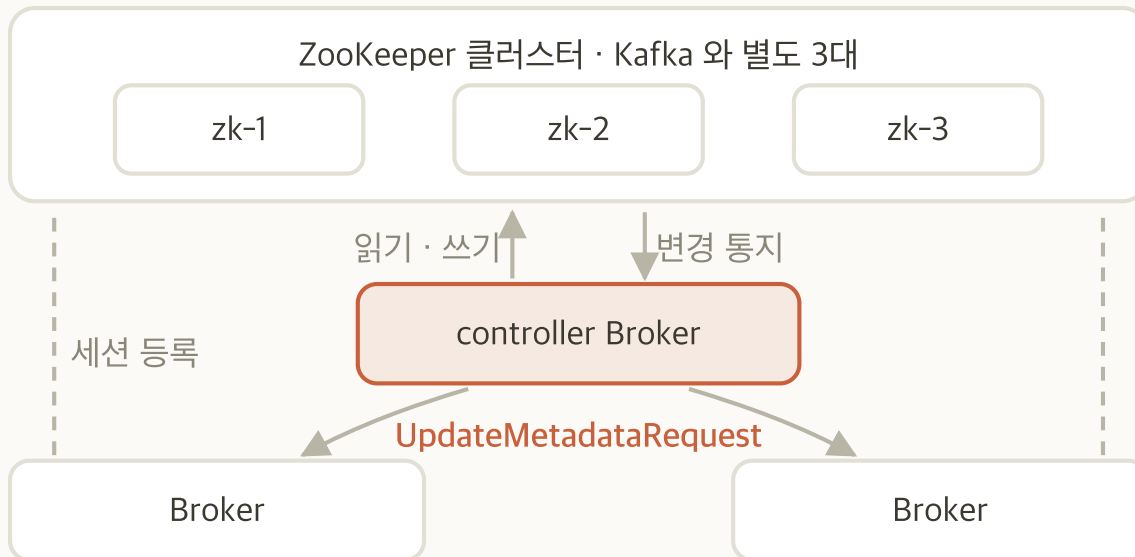
- **정의:** Topic 목록, Partition 배치, 각 Partition 의 leader 와 복제본, Broker 목록
- **합의가 필요한 이유:** Broker 마다 다르게 알고 있으면 클라이언트가 잘못된 Broker 로 요청을 보냄
- **저장 위치:** `__cluster_metadata` 라는 내부 Topic. **메타데이터도 로그**
- **쓰는 주체:** controller 중 leader 하나만 씀. 나머지는 그 로그를 따라 읽음

메타데이터가 멈추면 데이터는 흘러도 구성이 바뀌지 않습니다.

Topic 생성, Partition 재배포, leader 교체가 전부 멈춥니다.

ZooKeeper 의 역할

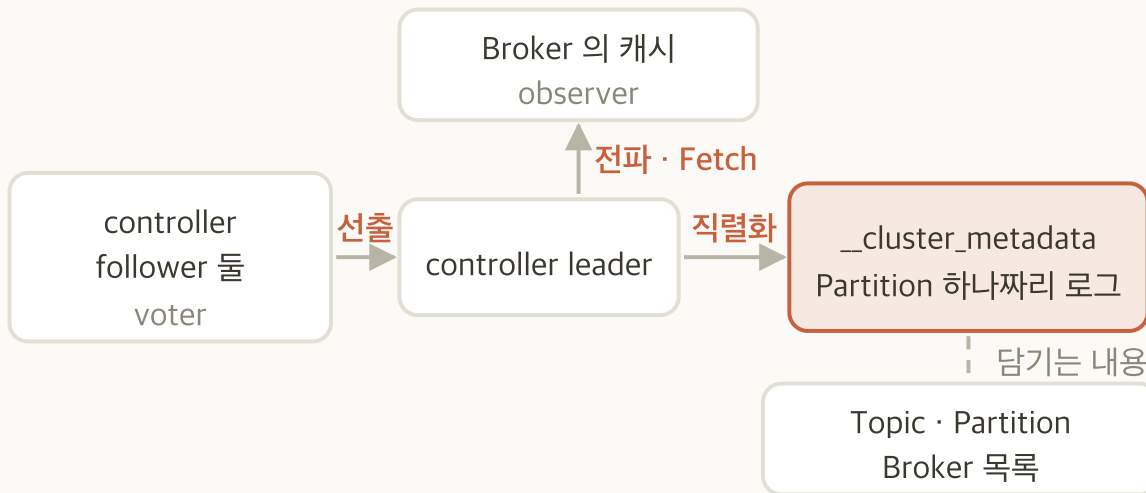
4.0 이전에는 Kafka 와 별도로 ZooKeeper 클러스터를 함께 운영했습니다. ZooKeeper 가 맡던 일을 알아야 KRaft 가 무엇을 대신하게 됐는지 이해할 수 있습니다.



- **메타데이터 저장:** Topic·Partition·설정을 ZooKeeper 에 두었음
- **controller 선출:** 어느 Broker 가 controller 가 될지 ZooKeeper 가 정했음
- **Broker 등록과 감시:** 세션이 끊기면 등록이 사라지고 ZooKeeper 가 controller 에 알렸음
- **Broker 는 ZooKeeper 를 직접 읽지 않았음:** controller 가 Broker 마다 따로 전달했음
- **운영 대상이 둘이었음:** 버전 관리와 백업, 모니터링, 장애 대응을 두 벌로 해야 했음

KRaft 의 정의와 역할

KRaft 는 Kafka Raft 입니다. Raft 는 여러 노드가 하나의 값에 합의하는 알고리즘이고, KRaft 는 그것을 Kafka 자신의 로그 위에 구현한 것입니다.



- **선출**: controller 들이 투표해 leader 하나를 뽑음
- **직렬화**: leader 만 씀. Offset 이 변경의 순서를 정함
- **저장**: 로그의 이름은 `__cluster_metadata`. Topic 목록, Partition 배치, Broker 목록이 담김
- **전파**: Broker 는 **leader 에게서** 로그를 `Fetch` 로 당겨 읽어 자기 캐시를 채움

controller follower 는 가용성을 위한 것이고, 실제 요청 처리에 쓰이는 것은 Broker 의 메타데이터 캐시입니다.

follower 는 투표에 참여하는 voter 이며, leader 가 정지하면 그중 하나가 역할을 이어받습니다. Broker 는 투표하지 않는 observer 로, leader 에게서 같은 로그를 받아 어느 Partition 이 어디에 있는지를 파악합니다.

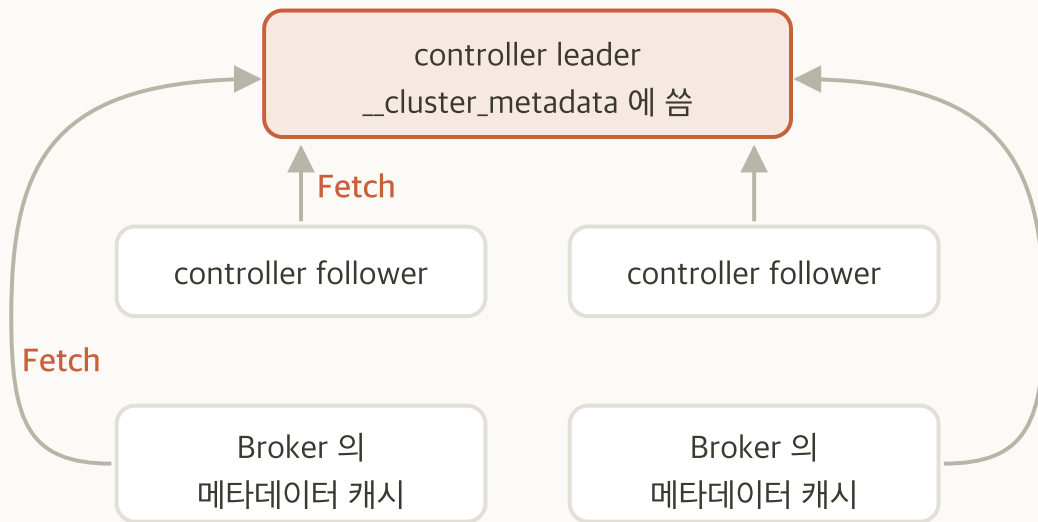
ZooKeeper 에서 KRaft 로

항목	ZooKeeper 시절	KRaft
메타데이터 저장소	별도 클러스터	Kafka 자신의 로그
운영 대상	Kafka 와 ZooKeeper 둘	Kafka 하나
메타데이터 전파	controller 가 Broker 에 개별 통지	Broker 가 로그를 따라 읽음
대규모에서	Partition 수가 늘면 통지가 병목	로그 복제라 같은 방식으로 확장

4.0에서 ZooKeeper 가 제거됐습니다.

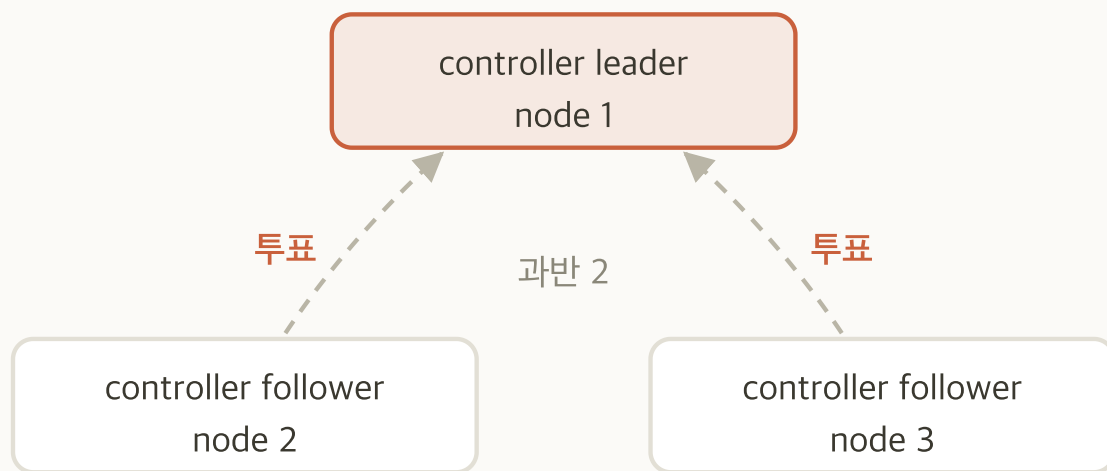
운영 대상을 하나로 줄이고, Partition 수가 늘어도 메타데이터 전파가 병목이 되지 않게 하기 위해서입니다.

KRaft 의 구조



- 쓰는 곳은 하나: controller leader 만
__cluster_metadata 에 씀
- 나머지는 당겨 읽음: follower 도 Broker 도 Fetch 로 같은 로 그를 순서대로 읽음
- 통지가 아니라 복제: ZooKeeper 시절의 개별 통지와 갈리는 지 점
- Broker 는 투표하지 않음: 메타데이터를 읽기만 하는 observer

controller quorum



- **과반 필요:** 3대 구성에서 2
- **leader 는 하나:** `ActiveControllerCount` 가 정확히 10이어야 함
- **1대가 정지해도 견딜:** 남은 2대가 과반이라 메타데이터 변경이 계속됨
- **2대가 정지하면 멈춤:** 과반을 못 채워 새 leader 를 뽑지 못함

combined 모드와 isolated 모드

항목	combined (이 실습)	isolated
프로세스	Broker 와 controller 가 한 프로세스	서로 다른 프로세스
노드 수	Broker 3대면 3대	Broker 3대 + controller 3대 = 6대
장애 영향	Broker 가 정지하면 controller 도 함께 정지	서로 독립적
롤링과 증설	controller 만 따로 롤링·증설할 수 없음	따로 롤링·증설할 수 있음
권장	개발과 소규모에 한함	운영 권장

Apache Kafka 는 운영 환경에서 combined 모드를 권장하지 않습니다.

controller 가 나머지와 격리되지 않고, controller 를 Broker 와 따로 롤링하거나 늘릴 수 없기 때문입니다. 이 실습이 combined 인 것은 분리하면 controller 전용 노드 세 대가 더 필요해서입니다.

내부 Topic

Topic	내용
<code>__cluster_metadata</code>	클러스터 메타데이터. KRaft 의 로그
<code>__consumer_offsets</code>	Consumer Group 의 Offset
<code>__transaction_state</code>	트랜잭션 상태. 이 과정의 범위 밖

- `cleanup.policy=compact` : 키별 최신 값만 남김
- 운영 대상: 내부 Topic 도 디스크를 쓰고 복제됨

자기 상태를 자기 Topic 에 담는 것은 이미 가진 로그 메커니즘을 그대로 쓰기 위해서입니다.

별도 저장소를 만들면 순서 보장과 복제, 따라잡기를 새로 구현해야 합니다. 로그에 담으면 Kafka 가 이미 하고 있는 방식이 그대로 적용됩니다.

클러스터 상태 확인

ops 노드에서 실행합니다.

- 1 `kafka-metadata-quorum.sh --bootstrap-server 10.0.1.5:9092 describe --status` 로 quorum 상태 확인
- 2 출력에서 `LeaderId` 와 `CurrentVoters` 기록
- 3 `kafka-metadata-quorum.sh --bootstrap-server 10.0.1.5:9092 describe --replication` 으로 세 노드의 복제 지연 확인
- 4 `kafka-broker-api-versions.sh --bootstrap-server 10.0.1.5:9092 | grep "id:"` 로 Broker 3대 확인

describe --status 출력

```
ClusterId:      A4lrm9UeR0qyJfRZk0xjDw   LeaderId:  1   LeaderEpoch:  1
HighWatermark:  2614   MaxFollowerLag:  0   MaxFollowerLagTimeMs:  0
CurrentVoters:  [{"id": 1, "endpoints": ["CONTROLLER://10.0.1.5:9093"]}, ...]
CurrentObservers:  []
```

필드	뜻	판정
LeaderId	지금 controller leader 의 node.id	값이 있어야 함. 비어 있으면 leader 가 없는 상태
LeaderEpoch	leader 가 바뀐 횟수	오르면 controller 가 교체된 것
CurrentVoters	투표권이 있는 노드	세 노드가 모두 있어야 함
CurrentObservers	투표권 없이 읽기만 하는 노드	비어 있음. combined 모드에서는 같은 노드가 이미 voter

07

복제와 ISR

leader 와 follower



- 읽기와 쓰기는 leader 가 받음: follower 는 복제만 함
- follower 증설의 효과: 처리량은 오르지 않고 내구성만 오름
- ISR (In-Sync Replica) 은 leader 를 따라잡은 복제본의 집합: leader 자신도 포함됨

Partition 마다 leader 가 따로 있습니다.

`lab.orders` 는 Partition 이 6개이므로 leader 도 6개이고, 세 Broker 에 나뉘어 있습니다.

replication.factor

- **정의:** Partition 하나를 몇 개의 Broker에 둘지 정하는 값. leader를 포함한 수
- **상한:** Broker 수. 복제본은 서로 다른 Broker에 놓임
- **표준값:** 3. 1대가 정지해도 복제본이 둘 남음
- **바꿀 수 있는가:** 만든 뒤에는 Partition 재배치로만 바꿈. `--alter` 로는 되지 않음

ISR

항목	내용	운영상 의미
정의	leader 를 충분히 따라잡은 복제본의 집합	leader 자신도 포함됨
판정 기준	<code>replica.lag.time.max.ms</code> 안에 leader 를 따라왔는가	시간 기준. 건수 기준이 아님
빠지는 순간	그 시간 동안 못 따라오면 제외됨	장애의 첫 신호
돌아오는 순간	다시 따라잡으면 자동으로 들어옴	사람이 넣지 않음
지표	ISR 수가 <code>replication.factor</code> 보다 작은 Partition 의 수	UnderReplicatedPartitions

ISR 에서 빠지는 것 자체는 오류가 아닙니다.

Broker 가 느려지거나 네트워크가 지연돼도 빠집니다. 계속 빠져 있는 것이 문제입니다.

acks

값	언제 응답하는가	유실 범위
0	보내고 바로	Producer 가 응답을 받지 않음. Broker 가 정지해도 알지 못함
1	leader 가 자기 로그에 쓴 뒤	leader 가 복제 전에 정지하면 그 메시지가 사라짐
all	ISR 전체가 받은 뒤	ISR 이 전부 정지하지 않는 한 남음

`acks` 는 Producer 가 정하고 `min.insync.replicas` 는 Topic 이 정합니다.

둘이 짝을 이뤄야 내구성이 성립합니다. 한쪽만으로는 의미가 없습니다.

min.insync.replicas 와 acks=all

ISR 수	min.insync.replicas	produce 결과
3	2	성공. 세 곳이 받음
2	2	성공. 두 곳이 받음
1	2	실패. NotEnoughReplicas
3	3	성공. 세 곳이 다 받아야 함
2	3	실패. Broker 1대만 빠져도 멈춤

min.insync.replicas 는 「몇 곳이 받아야 성공으로 볼 것인가」입니다.

이 값에 못 미치면 Broker 가 쓰기를 거부합니다. 조용히 복제가 덜 되는 것을 방지하기 위함입니다.

RF · acks · min.insync 조합

조합	무엇을 얻는가	결과
RF=3 , acks=1	복제본은 셋인데 leader 만 기다림	RF 가 의미 없음. leader 가 정지하면 유실
RF=3 , acks=all , min.insync=1	ISR 이 10이어도 성공	Broker 둘이 빠져도 계속 씬. 유실 위험이 큼
RF=3 , acks=all , min.insync=2	두 곳 이상이 받아야 성공	가용성과 내구성의 균형점. 이 실습에서 맞출 값
RF=3 , acks=all , min.insync=3	세 곳이 다 받아야 성공	가장 안전하지만 1대만 빠져도 멈춤

RF 만 3으로 두고 acks=1 이면 복제본은 백업일 뿐 내구성이 아닙니다.

Broker 정지와 produce

상황	ISR	produce
정상	3	성공
Broker 1대 정지	2	성공. <code>min.insync.replicas=2</code> 를 만족
Broker 2대 정지	1	실패. <code>NotEnoughReplicas</code>

Broker 가 3대여야 이 관찰이 성립합니다.

2대면 1대만 정지해도 곧바로 produce 가 실패해 볼 구간이 없습니다.

acks 와 min.insync.replicas 확인

- 1 ops 노드에서. `kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic lab.orders | grep Isr` 로 모든 Partition 의 `Isr` 이 3인지 확인
- 2 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config min.insync.replicas=2` 후 `--describe --entity-type topics --entity-name lab.orders` 로 Topic 에 값이 걸렸는지 확인
- 3 `kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --command-property acks=1` 로 한 줄, 이어서 `acks=all` 로 한 줄 produce
- 4 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config min.insync.replicas=3` 후 `acks=all` 로 다시 produce. **ISR 이 3이므로 성공합니다**
- 5 `kafka-configs.sh --bootstrap-server 10.0.1.5:9092 --alter --entity-type topics --entity-name lab.orders --add-config min.insync.replicas=2` 로 되돌리고 `--describe` 로 확인

kafka-configs.sh --describe 출력

```
Dynamic configs for topic lab.orders are:
  min.insync.replicas=2 sensitive=false
  synonyms={DYNAMIC_TOPIC_CONFIG:min.insync.replicas=2,
    DYNAMIC_DEFAULT_BROKER_CONFIG:...=1, DEFAULT_CONFIG:...=1}
```

- **DYNAMIC_TOPIC_CONFIG** : Topic 에 걸어 둔 값. 이 값이 우선함
- **DYNAMIC_DEFAULT_BROKER_CONFIG** · **DEFAULT_CONFIG** : Broker 에 동적으로 건 값과 Broker 기본값. 여기서는 둘 다 1
- **synonyms** : 같은 설정의 출처를 우선순위 순으로 보여 줌

Broker 기본값을 고치고 Topic 설정을 보면 바뀌지 않은 것처럼 보입니다.

Topic 에 값이 걸려 있으면 그 값이 우선합니다. 어느 층의 값을 보고 있는지 먼저 가립니다.

08

Broker 정지와 복귀

이 실습의 범위

- **미리 알려 주고 시작**: 무엇을 정지시킬지와 무엇이 일어나야 하는지를 먼저 안내함
- **원인을 찾는 실습이 아님**: 구조가 문서대로 동작하는지 관찰함
- **Broker 1대만 정지**: 2대를 정지시키면 produce 가 실패함

시작 전에 `lab.orders` 의 `min.insync.replicas` 가 2인지 확인합니다.

3인 상태에서 시작하면 Broker 1대만 정지시켜도 produce 가 실패합니다. 「`acks` 와 `min.insync.replicas` 확인」의 마지막 단계에서 2로 되돌렸는지 먼저 봅니다.

Broker 1대 정지

```
# ops 노드에서. kafka-2 를 정지
ansible kafka-2 -m systemd -a "name=kafka state=stopped" --become
# 관찰
kafka-topics.sh --bootstrap-server 10.0.1.5:9092 --describe --topic lab.orders
kafka-console-producer.sh --bootstrap-server 10.0.1.5:9092 --topic lab.orders --command-property acks=all
```

관찰 항목	예상 반응	판정
ISR	정지한 Broker 가 ISR 에서 제외	Isr 이 3개에서 2개로
leader 배치	그 Broker 가 leader 였던 Partition 만 교체	Leader 가 남은 Broker 로 변경
produce	성공	ISR 2개가 min.insync.replicas=2 를 만족
controller quorum	과반 2로 유지	LeaderId 가 여전히 하나
Grafana	복제 부족 Partition 발생	UnderReplicatedPartitions 가 0이 아님

ISR 이 줄어든 상태

```
Topic: lab.orders PartitionCount: 6 ReplicationFactor: 3 Configs: min.insync.replicas=2,...
Topic: lab.orders Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: LastKnownElr:
Topic: lab.orders Partition: 1 Leader: 3 Replicas: 2,3,1 Isr: 3,1
... Partition 2·3·4·5 도 Isr 에서 2가 빠졌습니다
```

- **Replicas** 는 그대로: 복제본을 둘 자리는 여전히 셋
- **Isr** 에서 2가 빠짐: 따라오지 못하는 복제본
- 일부 Partition 의 **Leader** 가 바뀜: 2가 leader 였던 Partition

Replicas 와 **Isr** 이 갈리는 것이 이 실습의 관찰 대상입니다.

정상 상태에서는 둘이 같습니다.

Broker 복귀

```
ansible kafka-2 -m systemd -a "name=kafka state=started" --become # ops 노드에서
```

관찰 항목	예상 반응	판정
ISR	follower 로 합류해 따라잡기 시작	Isr 이 2개에서 3개로
Grafana	복제 부족 Partition 해소	UnderReplicatedPartitions 가 0으로 복귀
leader 배치	즉시 돌아오지 않음	정지 전과 배치가 다를 수 있음

Broker 가 돌아와도 leader 배치는 곧바로 원래대로 돌아오지 않습니다. `auto.leader.rebalance.enable` 이 이 실습에서는 `false` 입니다. 위 관찰이 끝난 뒤 `kafka-leader-election.sh --bootstrap-server 10.0.1.5:9092 --election-type preferred --all-topic-partitions` 로 배치를 되돌립니다.

개념과 관찰 결과의 대응

개념	관찰 결과
따라잡은 복제본의 집합인 ISR	Isr 에서 한 Broker 가 빠짐
Partition 마다 따로 있는 leader	정지한 Broker 가 맡던 Partition 만 leader 가 바뀜
<code>min.insync.replicas=2</code> 에서 1대 정지 시 쓰기 유지	<code>acks=all</code> produce 가 성공
과반이 2인 quorum	LeaderId 가 유지됨
복귀 뒤에도 유지되는 leader 배치	선호 leader 선출을 실행하기 전까지 배치가 그대로였음

리소스 정리

```
# 실습 PC 의 lab/terraform/03-kafka 에서 실행. ops 노드 아님
terraform destroy -var-file=session.tfvars
```

- 포털에서 리소스 그룹이 실제로 사라졌는지 확인. 한 번에 끝나지 않는 경우가 있음
- 세션 간에 자원을 넘기지 않음. 다음 세션은 04-operation 을 새로 만듦
- 다음 세션은 Redis·MongoDB·Kafka 를 모두 새로 띄움. 이 세션의 Kafka 를 남기지 않음

destroy 는 리소스 그룹과 그 안의 자원을 전부 삭제합니다.

되돌릴 수 없습니다. **describe** 출력과 설정값 기록 중 남길 것을 먼저 실습 PC 로 옮기세요.